

Full-Text Searching

In this chapter, you'll learn how to use MySQL's full-text searching capabilities to perform sophisticated data querying and selection.

Understanding Full-Text Searching

Note

Not All Engines Support Full-Text Searching As will be explained in Chapter 21, “Creating and Manipulating Tables,” MySQL supports the use of several underlying database engines. Not all engines support full-text searching as described in this chapter. The two most commonly used engines are MyISAM and InnoDB; the former supports full-text searching, and the latter does not. This is why although most of the sample tables used in this book were created to use InnoDB, one table (the `productnotes` table) was created to use MyISAM. If you need full-text searching functionality in your applications, keep this in mind.

In Chapter 8, “Using Wildcard Filtering,” you were introduced to the `LIKE` keyword that is used to match text (and partial text) using wildcard operators. By using `LIKE`, you can locate rows that contain specific values or parts of values, regardless of the location of those values within the columns.

In Chapter 9, “Searching Using Regular Expressions,” we took text-based searching one step further by using regular expressions to match column values. By using regular expressions, it is possible to write very sophisticated matching patterns to locate the desired rows.

But as useful as these search mechanisms are, they have several very important limitations:

- **Performance:** Wildcard and regular expression matching usually requires that MySQL try to match each and every row in a table (and table indexes are rarely of use in these searches). These searches can therefore be very time-consuming as the number of rows to be searched grows.

- **Explicit control:** Using wildcard and regular expression matching, it is very difficult (and not always possible) to explicitly control what is and what is not matched. An example of this is a search specifying a word that must be matched, a word that must not be matched, and a word that may or may not be matched but only if the first word is indeed matched.
- **Intelligent results:** Although wildcard- and regular expression-based searching provide for very flexible searching, neither provides an intelligent way to select results. For example, searching for a specific word would return all rows that contain that word and would not distinguish between rows that contain a single match and those that contain multiple matches (ranking them as potentially better matches). Similarly, searches for a specific word would not find rows that do not contain that word but that do contain other related words.

All of these limitations and more are addressed by full-text searching. When full-text searching is used, MySQL does not need to look at each row individually, analyzing and processing each word individually. Rather, MySQL creates an index of the words (in specified columns), and searches can be made against those words. MySQL can thus quickly and efficiently determine which words match (that is, which rows contain them), which don't, how often they match, and so on.

Using Full-Text Searching

Full-text searching can only be performed on tables with columns specifically indexed to be searchable in this way. Index creation is generally done at table creation time, and the `productnotes` table has a `note_text` column that we've created for just this purpose.

Performing Full-Text Searches

After indexing, full-text searches are performed using two functions: `Match()` to specify the columns to be searched and `Against()` to specify the search expression to be used.

Here is a basic example:

► Input

```
SELECT note_text
FROM productnotes
WHERE Match(note_text) Against('rabbit');
```

► Output

```
+-----+
| note_text |
+-----+
| Customer complaint: rabbit has been able to detect trap, food apparently less effective now. |
| Quantity varies, sold by the sack load. All guaranteed to be bright and orange, and suitable for use as rabbit bait. |
+-----+
```

► Analysis

In this example, the `SELECT` statement retrieves a single column, `note_text`. For the `WHERE` clause, a full-text search is performed. `Match(note_text)` instructs MySQL to perform the search against that named column, and `Against('rabbit')` specifies the word `rabbit` as the search text. The two rows that contain the word `rabbit` are returned.

Note

Using the Full `Match()` Specification The value passed to `Match()` must be the same as the one used in the `FULLTEXT()` definition. If multiple columns are specified, all of them must be listed (and in the correct order).

Note

Searches Are Not Case-Sensitive Full-text searches are not case-sensitive unless `BINARY` mode (not covered in this chapter) is used.

The search just performed could just as easily use a `LIKE` clause, as shown here:

► Input

```
SELECT note_text
FROM productnotes
WHERE note_text LIKE '%rabbit%';
```

► Output

```
+-----+
| note_text                                     |
+-----+
| Quantity varies, sold by the sack load. All guaranteed to be |
| bright and orange, and suitable for use as rabbit bait.      |
| Customer complaint: rabbit has been able to detect trap, food |
| apparently less effective now.                               |
+-----+
```

► Analysis

This `SELECT` retrieves the same two rows, but the order is different (although that may not always be the case).

Neither of the two `SELECT` statements contains an `ORDER BY` clause. The latter (using `LIKE`) returns data in no particularly useful order. But the former (using full-text searching) returns data ordered by how well the text matched. Both rows contain the word `rabbit`, but the row that contains the word `rabbit` as the 3rd word ranks higher than the row that contains it as the 20th word. This is important. An important part of full-text searching is the ranking of results. Rows with a higher rank are returned first (as there is a higher degree of likelihood that those are the rows you really want).

To see how ranking works, look at this example:

► Input

```
SELECT note_text,  
       Match(note_text) Against('rabbit') AS match_rank  
FROM productnotes;
```

► Output

note_text	match_rank
Customer complaint: Sticks not individually wrapped, too easy to mistakenly detonate all at once. Recommend individual wrapping.	0
Can shipped full, refills not available. Need to order new can if refill needed.	0
Safe is combination locked, combination not provided with safe. This is rarely a problem as safes are typically blown up or dropped by customers.	0
Quantity varies, sold by the sack load. All guaranteed to be bright and orange, and suitable for as rabbit bait.	1.5905543170914
Included fuses are short and have been known to detonate too quickly for some customers. Longer fuses are available (item FU1) and should be recommended.	0
Matches not included, recommend purchase of matches or detonator (item DTNTR).	0
Please note that no returns will be accepted if safe opened using explosives.	0
Multiple customer returns, anvils failing to drop fast enough or falling backwards on purchaser. Recommend that customer considers using heavier anvils.	0
Item is extremely heavy. Designed for dropping, not recommended for use with slings, ropes, pulleys, or tightropes.	0
Customer complaint: rabbit has been able to detect trap, food apparently less effective now.	1.6408053837485
Shipped unassembled, requires common tools (including oversized hammer).	0
Customer complaint: Circular hole in safe floor can apparently be easily cut with handsaw.	0
Customer complaint: Not heavy enough to generate flying stars around head of victim.	0
If being purchased for dropping, recommend ANV02 or ANV03 instead.	

Call from individual trapped in safe plummeting	0	
to the ground, suggests an escape hatch be		
added. Comment forwarded to vendor.		
+-----+-----+		

► Analysis

Here `Match()` and `Against()` are used in the `SELECT` instead of the `WHERE` clause. This causes all rows to be returned (as there is no `WHERE` clause). `Match()` and `Against()` are used to create a calculated column (with the alias `match_rank`) that contains the ranking value calculated by the full-text search. MySQL calculates the ranking based on the number of words in the row, the number of unique words, the total number of words in the entire index, and the number of rows that contain the word.

As you can see, the rows that do not contain the word `rabbit` have a rank of 0 (and were therefore not selected by the `WHERE` clause in the previous example). The two rows that do contain the word `rabbit` each have a rank value, and the one with the word earlier in the text has a higher rank value than the one in which the word appears later. This helps demonstrate how full-text searching eliminates rows (those with a rank of 0), and how it sorts results (by rank, in descending order).

Note

Ranking Multiple Search Terms If multiple search terms are specified, those that contain the most matching words are ranked higher than those with fewer (or with just a single match).

As you can see, full-text searching offers functionality not available with simple `LIKE` searches. And as data is indexed, full-text searches are considerably faster, too.

Using Query Expansion

Query expansion is used to try to widen the range of returned full-text search results. Consider the following scenario. You want to find all notes with references to `anvils` in them. Only one note contains the word `anvils`, but you also want any other rows that may be related to your search, even if the specific word `anvils` is not contained within them.

This is a job for query expansion. When query expansion is used, MySQL makes two passes through the data and indexes to perform the search:

1. First, MySQL performs a basic full-text search to find all rows that match the search criteria.
2. MySQL examines those matched rows and selects all useful words. (We'll talk about how MySQL figures out what is useful and what is not shortly.)
3. MySQL performs the full-text search again, this time using not just the original criteria but also all of the useful words.

So, by using query expansion, you can find results that might be relevant, even if they don't contain the exact words for which you are looking.

Let's look at an example. First, a simple full-text search, without query expansion:

► Input

```
SELECT note_text
FROM productnotes
WHERE Match(note_text) Against('anvils');
```

► Output

```
+-----+
| note_text |
+-----+
| Multiple customer returns, anvils failing to drop fast enough or |
| falling backwards on purchaser. Recommend that customer considers |
| using heavier anvils. |
+-----+
```

► Analysis

Only one row contains the word *anvils*, so only one row is returned.
Here is the same search, this time using query expansion:

► Input

```
SELECT note_text
FROM productnotes
WHERE Match(note_text)
      Against('anvils' WITH QUERY EXPANSION);
```

► Output

```
+-----+
| note_text |
+-----+
| Multiple customer returns, anvils failing to drop fast enough or |
| falling backwards on purchaser. Recommend that customer considers |
| using heavier anvils. |
| Customer complaint: Sticks not individually wrapped, too easy to |
| mistakenly detonate all at once. Recommend individual wrapping. |
| Customer complaint: Not heavy enough to generate flying stars |
| around head of victim. If being purchased for dropping, recommend |
| ANV02 or ANV03 instead. |
| Please note that no returns will be accepted if safe opened using |
| explosives. |
| Customer complaint: rabbit has been able to detect trap, food |
| apparently less effective now. |
| Customer complaint: Circular hole in safe floor can apparently be |
| easily cut with handsaw. |
| Matches not included, recommend purchase of matches or detonator |
| (item DTNTR). |
+-----+
```

► Analysis

This time, seven rows are returned. The first row contains the word `anvils` and is thus ranked highest. The second row has nothing to do with `anvils`, but because it contains two words that are also in the first row (`customer` and `recommend`), it is retrieved, too. The third row also contains those same two words, but they are further into the text and further apart, and so this row is included but ranked third. And this third row does indeed refer to `anvils` (by their product name).

As you can see, query expansion greatly increases the number of rows returned, but in doing so, it also increases the number of items returned that you might not actually want.

Tip

The More Rows, the Better The more rows in your table (and the more text within those rows), the better the results returned when using query expansion.

Boolean Text Searches

MySQL supports an additional form of full-text searching called *Boolean mode*. In Boolean mode, you can provide specifics about a number of things, including the following:

- Words to be matched
- Words to be excluded (so that if a row contains this word, it will not be returned, even though other specified words are matched)
- Ranking hints (specifying which words are more important than others so they can be ranked higher)
- Expression grouping

Tip

Usable Even Without a FULLTEXT Index Boolean mode differs from the full-text search syntax used thus far in that it may be used even if no `FULLTEXT` index is defined. However, this would be a very slow operation, and the performance would degrade further as data volume increased.

To demonstrate what Boolean mode does, here is a simple example:

► Input

```
SELECT note_text
FROM productnotes
WHERE Match(note_text)
      Against('heavy' IN BOOLEAN MODE);
```

► Output

```
+-----+
| note_text |
+-----+
| Item is extremely heavy. Designed for dropping, not recommended |
| for use with slings, ropes, pulleys, or tightropes. |
| Customer complaint: Not heavy enough to generate flying stars |
| around head of victim. If being purchased for dropping, recommend |
| ANV02 or ANV03 instead. |
+-----+
```

► Analysis

This full-text search retrieves all rows containing the word *heavy* (there are two of them). The keywords `IN BOOLEAN MODE` are specified, but no Boolean operators are actually specified, and so the results are the same as if Boolean mode were not specified.

Note

IN BOOLEAN MODE Behaves Differently Although the results in this example are the same as they would be without `IN BOOLEAN MODE`, there is an important difference in behavior (even if it does not manifest itself in this particular example). I'll point out the difference later in this chapter.

To match the rows that contain *heavy* but not any word beginning with *rope*, you can use the following:

► Input

```
SELECT note_text
FROM productnotes
WHERE Match(note_text)
      Against('heavy -rope*' IN BOOLEAN MODE);
```

► Output

```
+-----+
| note_text |
+-----+
| Customer complaint: Not heavy enough to generate flying stars |
| around head of victim. If being purchased for dropping, recommend |
| ANV02 or ANV03 instead. |
+-----+
```

► Analysis

This time, only one row is returned. Again, the word *heavy* is matched, but this time `-rope*` instructs MySQL to explicitly exclude any row that contains *rope** (that is, any word that begins with *rope*, including *ropes*, which is why one of the rows was excluded).

You have now seen two full-text search Boolean operators: - excludes a word and * is the truncation operator (which you can think of as a wildcard used at the end of a word). Table 18.1 lists all of the supported Boolean operators.

TABLE 18.1 Full-Text Boolean Operators

Operator	Description
+	Include the specified word.
-	Exclude the specified word.
>	Include the specified word and increase its ranking value.
<	Include the specified word and decrease its ranking value.
()	Group the specified words into subexpressions (allowing them to be included, excluded, ranked, and so forth as a group).
~	Negate the specified word's ranking value.
*	Use as a wildcard at the end of a word.
" "	Use a text phrase (as opposed to a list of individual words) to match for inclusion or exclusion.

Here are some more examples to demonstrate the use of some of these operators:

► Input

```
SELECT note_text
FROM productnotes
WHERE Match(note_text)
      Against('+rabbit +bait' IN BOOLEAN MODE);
```

► Analysis

This search matches rows that contain both the words `rabbit` and `bait`.

► Input

```
SELECT note_text
FROM productnotes
WHERE Match(note_text)
      Against('rabbit bait' IN BOOLEAN MODE);
```

► Analysis

Without operators specified, this search matches rows that contain at least one instance of `rabbit` or `bait`.

► Input

```
SELECT note_text
FROM productnotes
WHERE Match(note_text)
      Against('"rabbit bait"' IN BOOLEAN MODE);
```

► Analysis

This search matches the phrase `rabbit bait` instead of the two words `rabbit` and `bait`.

► Input

```
SELECT note_text
FROM productnotes
WHERE Match(note_text)
      Against('>rabbit <carrot' IN BOOLEAN MODE);
```

► Analysis

This search matches both `rabbit` and `carrot`, increasing the rank of the former and decreasing the rank of the latter.

► Input

```
SELECT note_text
FROM productnotes
WHERE Match(note_text)
      Against('+safe +(<combination)' IN BOOLEAN MODE);
```

► Analysis

This search matches the words `safe` and `combination`, lowering the ranking of the latter.

Note

Ranked but Not Sorted In Boolean mode, rows are returned ranked in descending order by score but not sorted.

Full-Text Searching Notes

Before finishing this chapter, here are some important notes pertaining to the use of full-text searching:

- When indexing full-text data, short words are ignored and are excluded from the index. Short words are by default defined as those having three or fewer characters (though this number can be changed, if needed).
- MySQL comes with a built-in list of *stopwords*, which are words that are always ignored when indexing full-text data. This list can be overridden, if needed. (Refer to the MySQL documentation to learn how to accomplish this.)
- Many words appear so frequently that searching on them would be useless (that is, too many results would be returned). To deal with this, MySQL follows a 50% rule: If a word appears in 50% or more rows, it is treated as a stopword and is effectively ignored. (The 50% rule is not used for Boolean mode.)
- Full-text searching never returns any results if there are fewer than three rows in a table (because every word is always in at least 50% of the rows).

- Single quote characters in words are ignored. For example, `don't` is indexed as `dont`.
- Languages that don't have word delimiters (including Japanese and Chinese) do not return full-text results properly.
- As already noted, full-text searching is only supported in the MyISAM database engine.

Note

No Proximity Operators One feature supported by many full-text search engines is proximity searching, which involves searching for words that are near each other (in the same sentence, in the same paragraph, no more than a specific number of words apart, and so on). Proximity operators are not yet supported for MySQL full-text searching, although this is planned for a future release.

Summary

In this chapter, you learned why full-text searching is used and how to use the MySQL `Match()` and `Against()` functions to perform these searches. You also learned about query expansion as a way to increase the chances of finding related matches and how to use Boolean mode for more granular lookup control.

Challenges

1. Write a SQL statement that uses full-text searching to return all rows that contain the word `safe` but that do not contain the word `handsaw`.
2. Write a SQL statement that uses full-text searching to return all rows that contain the words `drop`, `dropped`, `dropping`, and any other word that begins with `drop`.

This page intentionally left blank